

Cloud Computing– 2026/27

Project 1

Introduction

The goal of this project is to understand how services available in cloud computing platforms can be used for creating applications that are scalable, fast, and highly available, and what are the trade-offs of different services in terms of performance and cost.

The project will consist of 4 small parts. For each of these parts, students are supposed to produce some software deliverable and a small – 2 pages – report.

Students can deliver each part individually or all parts together. The final deadline will be on 2nd November 2026.

The deadline for each part will be one week after the lab the students are enrolled in. For groups with students in two different labs, the deadline is one week after the earliest lab. All parts can be delivered in the final deadline date, with a penalty a 10%.

Project base

The base project is provided to you. It is the backend for an auction system, like eBay, together with the companion testing scripts. It is a complete, working system: it implements all the operations described below and runs on Azure App Service.

The system manages auctions. Users create auctions and bid on open auctions; a bid is only accepted while the auction is open and before its end time, if it is at least the minimum price of the auction and higher than the current winning bid. Users can also ask questions about the product of an auction. A question can only be answered by the user that created the auction, and there can be only one answer for each question.

The system maintains the following information, stored in Cosmos DB, with one container for each type (you can add more information, as it seems appropriate for your system):

- **User:** the nickname, which is the id of the user, the name, the hash of the password, and the id of a photo;
- **Media:** images used in the system, identified by a hash of their contents;
- **Auction:** the title, the description, the id of an image, the owner (the user that created the auction), the end time of the auction (i.e., until when bids can be made), the minimum price, the winning bid, and the status of the auction (open, closed, deleted). An auction is always created open; the provided code does not close auctions when their end time passes;
- **Bid:** the auction it belongs to, the user that made the bid, the value of the bid, and the time it was made;
- **Question:** the auction it refers to, the user that asked it, the text of the question and, once answered, the text of the reply.

The system provides the following REST endpoints:

- **User** (/rest/user): create, retrieve, update and delete users; authenticate a user (/rest/user/auth); list the auctions of a user (/rest/user/{id}/auctions), optionally

only those with a given status. After a user is deleted, the auctions and bids of the user appear as having been made by a default “Deleted User” user.

- **Media** (/rest/media): upload an image, which returns its id; download an image.
- **Auction** (/rest/auction): create, retrieve, update and list auctions (no delete auction); list the most recent auctions (/rest/auction/any/recent), the auctions about to close (/rest/auction/any/about-to-close), and the most popular auctions, i.e., those with more bids (/rest/auction/any/popular); search auctions by the text of their title and description (/rest/auction/search?q=text).
- **Bid** (/rest/auction/{id}/bid): create bid, list bids, retrieve a bid.
- **Question** (/rest/auction/{id}/question): create question, list questions. Replies to a question are made using a POST in URL (/rest/auction/{id}/question/{question id}/reply).
- **Control** (/rest/ctrl/version): returns the version of the service.

Operations that return lists are paginated with two query parameters: st, the position of the first element (0 by default), and len, the number of elements (20 by default).

Operations performed on behalf of a user – creating or updating an auction, bidding, replying to a question, and updating or deleting a user – are only accepted from an authenticated user. /rest/user/auth checks the password of the user and returns a session cookie, valid for one hour; sessions are stored in the database, and each of these operations checks that the session belongs to the user on whose behalf it is performed.

Operations that create an entity with an image (users and auctions) take two consecutive calls: the client first uploads the image to /rest/media, and then passes the id returned in the corresponding operation.

The logic of the operations is independent of the REST framework. Package cc.srv contains the REST resources, which only read the parameters, call the operation and build the response; package cc.ops contains the logic of each operation – validation, access control and database access; packages cc.data and cc.db contain the data objects and the access to Cosmos DB. Operations in cc.ops do not throw exceptions: they return a Result, which carries either the value or an HTTP status code.

The following code is provided:

- **cc-proj (lab 1)**: the backend, deployed as a WAR to Azure App Service; and the Terraform configuration that creates the resources it needs – a Cosmos DB account, database and containers, and the App Service – and passes the database keys to the application as environment variables. cc.utils.TestBackend exercises every endpoint against a running instance.
- **cc-proj-fun (lab 1)**: the skeleton of an Azure Functions project, with a single HTTP function, and (lab 2) the Terraform configuration that creates the function app, reusing the database created for cc-proj.
- **cc-proj-test (lab 2)**: artillery scripts that populate the system with users and auctions, and a workload that exercises it.

The provided system works, but it is deliberately naive in a number of ways – for example, images are stored inside database documents, there is no cache, and listings and searches are computed from the database on every request. Each part of the project removes some

of these limitations and measures the effect on performance and cost. Do not fix them before the part that asks you to: you need the original version to compare with.

1.1 Environment

The project is implemented in Azure, using the resources available to students under the Azure for Students program. Check the documentation in lab 0/1 for how to claim the credits for your Azure account.

NOTE: Azure services have associated costs. Be careful with the configuration of your services and with the cost of leaving services/resources running, especially those that are more expensive, such as Redis. Delete the resources when you are not using them – “terraform destroy” removes everything that “terraform apply” created.

Cloud Computing– 2026/27

Project 1 – part 1

The provided system runs as a web application on Azure App Service. In this part, you implement a second version of the same service using Azure Functions, and compare the two versions in terms of performance and cost.

Tasks

- **Deploy the provided solution.** Create the resources with the Terraform configuration in cc-proj, deploy the application to App Service, and populate it using the scripts in cc-proj-test.
- **Move the code to Azure Functions.** In cc-proj-fun, implement the REST endpoints of the service as Azure Functions with HTTP triggers. Each function reads the parameters of the request, calls the corresponding operation in cc.ops, and builds the response from the Result it returns – the operations do not depend on the REST framework, so they can be reused unchanged. The functions must be available at similar URLs as in the App Service version (only the prefix can change), so that the same artillery scripts can be run against both, with a minimal change. Both versions use the same database (and other resources in the future).
- **Evaluate the performance using artillery.** Run the workloads in cc-proj-test (you can extend them) against both versions, and compare latency – e.g., median and 95th/99th percentiles per endpoint – and throughput.

For the comparison to be meaningful, run both versions on resources of similar capacity – e.g., use the B1 tier for App Service. For Azure Functions, do not use always-ready (pre-warmed) instances: the latency of the first requests after an idle period (the cold start) is part of what you are measuring. Run the artillery clients from the same place for both versions.

Deliverables

- **Report** – maximum: 3 pages (including graphs/tables) – including:
 - **Commit point:** the commit, in your group's repository, that contains the code for this part.
 - **Introduction:** give context to the reader. What are you doing? Why are you doing it (e.g., better cost, performance, ease of use, etc)? Give a brief summary of your findings.
 - **Design:** a brief explanation of the design and changes in the code, and of the differences between the two versions;
 - **Evaluation:** the methodology of experiments, the results obtained with artillery, and their discussion. Include your expectations, if they were met or not, and justify
 - **Cost:** a brief analysis of the cost of App Service versus Azure Functions – consider only these, since the database is the same for both versions – in two scenarios: low usage (e.g., a few requests per minute, with long idle periods) and high usage (e.g. hundreds of thousands of request per day).
 - **Conclusion:** a short recap what you did in this project. What would you do if you had more time (e.g. what aspect of your evaluation could be

improved; what experiments would you have run if you wanted to know more about X)?

Deadline

October 8th, 23h59

We will provide information about how to deliver the first part of the project.